10.9 Consistência e disponibilidade em presença de partições

Quando a rede se divide e dois lados continuam recebendo operações, o sistema precisa escolher que propriedades preservar. Alguns domínios preferem rejeitar alterações sem quorum para proteger consistência; outros aceitam alterações locais e reconciliam posteriormente para manter disponibilidade. Não existe resposta universal: saldo bancário, curtidas em rede social e telemetria podem ter necessidades diferentes.

10.10 Padrões de resiliência combinados

Tabela 19 - Padrões de resiliência e seus trade-offs

Padrão	Problema	Risco de uso incorreto
Timeout	Evitar espera indefinida	Curto demais causa falsos fracassos; longo demais ocupa recursos
Retry	Superar falha transitória	Duplicação e tempestade de tráfego
Backoff + jitter	Espalhar retries no tempo	Aumenta latência de recuperação se exagerado
Circuit breaker	Parar de pressionar dependência falha	Pode bloquear recuperação se política ruim
Bulkhead	Isolar recursos por fluxo	Subutilização e configuração complexa
Fallback	Oferecer resposta degradada	Pode esconder erro ou fornecer dado inadequado

10.11 Idempotency key

Uma estratégia para comandos não idempotentes é o cliente enviar uma chave única por intenção de negócio. O servidor registra a chave e o resultado; se receber a mesma chave novamente, devolve o resultado anterior em vez de repetir o efeito. Isso é comum em pagamentos e criação de recursos. A retenção da chave deve cobrir a janela de retry relevante e sua implementação precisa ser transacional com o efeito protegido.

10.12 Threat modeling simplificado

Antes de escolher controles, identifique ativos, fronteiras de confiança, agentes e possíveis abusos. Pergunte quem pode enviar mensagens, quem pode escutar o canal, quem controla o broker, que dados são sensíveis e o impacto de falsificação, alteração, negação de serviço ou vazamento. O objetivo não é listar todas as vulnerabilidades, mas conectar ameaças concretas a controles verificáveis.

11. Blockchain como Sistema Distribuído

Objetivos de aprendizagem

- Explicar blockchain como ledger replicado entre nós.
- Relacionar hash, encadeamento, validação, consenso e replicação.
- Distinguir integridade criptográfica de consenso.
- Comparar Proof of Work e Proof of Stake em nível conceitual.
- Analisar limites de descentralização, desempenho e segurança.

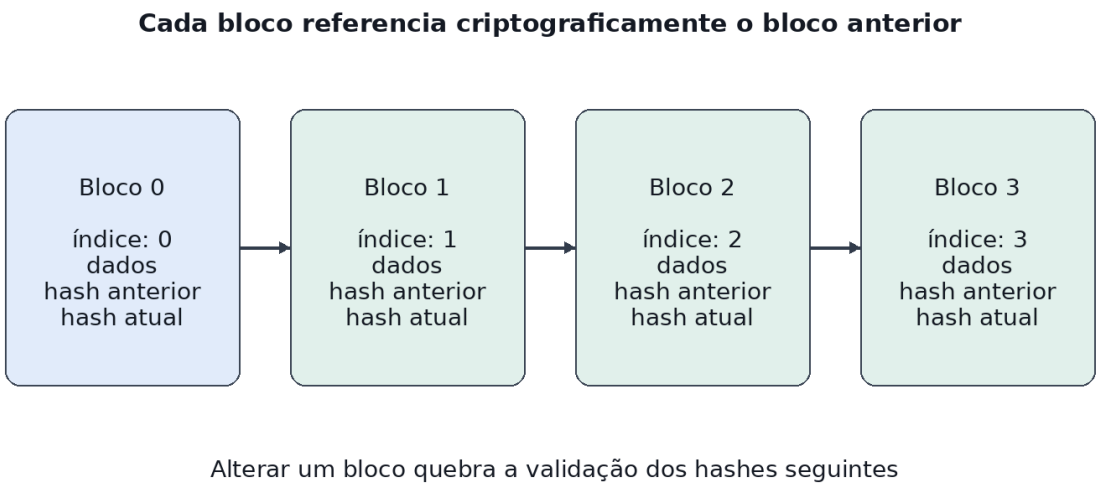


Figura 8 - Encadeamento criptográfico simplificado de blocos

11.1 Ledger replicado

Uma blockchain mantém um histórico de transações ou estados replicado entre múltiplos nós. O desafio não é apenas armazenar blocos, mas fazer participantes chegarem a um estado aceito apesar de concorrência, mensagens atrasadas, falhas e, em redes permissionless, participantes potencialmente adversários. Hashes fornecem integridade e ligação entre blocos; mecanismos de consenso definem como o sistema escolhe uma história canônica ou finaliza estados.

11.2 Hash criptográfico

Uma função hash mapeia dados de tamanho arbitrário para uma saída de tamanho fixo. Para fins de blockchain, propriedades desejáveis incluem resistência prática a preimagem, segunda preimagem e colisões. Pequena alteração no conteúdo tende a produzir hash completamente diferente. Isso torna adulterações detectáveis quando o hash é recalculado.

hash_demo.py

```
import hashlib

def sha256(texto: str) -> str:
    return hashlib.sha256(texto.encode("utf-8")).hexdigest()

print(sha256("bloco A"))
print(sha256("bloco a"))
```

11.3 Blocos, transações e Merkle trees

Blocos geralmente agregam transações e metadados. Em blockchains reais, estruturas como Merkle trees permitem resumir muitas transações em uma raiz criptográfica e produzir provas de inclusão eficientes. A blockchain didática da AP7 usa uma lista simples para focar encadeamento e validação; isso não pretende reproduzir todos os mecanismos de uma rede real.

11.4 Consenso e forks

Quando nós propõem blocos concorrentes, a rede precisa de uma regra para convergir. Em Proof of Work, participantes gastam trabalho computacional e a regra de seleção considera a cadeia com maior trabalho acumulado. Em Proof of Stake, validadores comprometem stake e seguem regras do

protocolo para propor/atestar blocos, com mecanismos econômicos e de finalidade. A comparação deve considerar segurança, energia, latência de finalização, incentivos e modelo de rede.

11.5 CAP, consenso e simplificações perigosas

É comum usar o teorema CAP de forma excessivamente simplificada. CAP trata do comportamento sob particionamento de rede e da impossibilidade de garantir simultaneamente certas propriedades fortes de consistência e disponibilidade em todas as situações. Não é uma classificação permanente de sistemas em “CP” ou “AP”, nem substitui análise de modelos de consistência, latência e requisitos de falha.

11.6 Segurança e confiança

Blockchain não elimina confiança; ela redistribui confiança entre protocolo, criptografia, software cliente, governança, validadores/mineradores e regras econômicas. Smart contracts podem conter bugs, chaves privadas podem ser comprometidas e bridges ou oráculos podem introduzir novos pontos de confiança.

11.7 Desempenho e trade-offs

Replicação global e consenso custam desempenho. Em muitos cenários empresariais, um banco de dados tradicional com controle de acesso atende melhor. Blockchain é justificável quando múltiplas partes precisam compartilhar estado e reduzir dependência de uma autoridade única, e aceitam custos de consenso, governança e operação.

11.8 Assinaturas digitais e autoria de transações

Hashes detectam alteração, mas não provam quem autorizou uma transação. Assinaturas digitais usam uma chave privada para produzir uma assinatura verificável por quem conhece a chave pública correspondente. Em redes blockchain, endereços e contas costumam estar associados a chaves, e uma transação válida precisa satisfazer regras de assinatura. Perder a chave privada pode significar perder capacidade de autorizar operações; roubá-la permite ao atacante agir como o titular.

11.9 Ciclo de vida de uma transação

De forma abstrata, uma transação é criada e assinada, propagada para nós, validada segundo regras locais, colocada em uma área de pendência, selecionada para um bloco, ordenada e finalmente incorporada à história aceita. Dependendo do protocolo, a finalidade pode ser probabilística ou explícita. Aplicações precisam decidir quantas confirmações ou qual condição de finalização é suficiente para seu risco.

11.10 Redes permissionless e permissionadas

Em rede permissionless, participação e validação tendem a ser abertas segundo regras públicas, exigindo mecanismos robustos contra identidades arbitrárias e adversários econômicos. Em rede permissionada, participantes são conhecidos ou admitidos por governança, permitindo protocolos e controles diferentes. Permissionada não significa automaticamente privada: visibilidade dos dados, governança e identidade são dimensões separadas.

11.11 Merkle proofs em perspectiva

Uma Merkle tree combina hashes em níveis até uma raiz. Para provar que uma transação pertence a um bloco, não é necessário transmitir todas as transações; basta fornecer os hashes irmãos ao longo do caminho. O verificador recalcula a raiz e compara com a raiz comprometida no cabeçalho. Essa

técnica ilustra como estruturas criptográficas podem reduzir volume de verificação sem confiar no provedor da prova.

11.12 Finalidade, confirmação e reorganização

Em protocolos com possibilidade de reorganização, um bloco observado pode deixar a cadeia canônica depois. Aplicações não devem tratar “incluído em um bloco” como sinônimo universal de irreversível. O requisito de negócio define tolerância: uma aplicação de baixo valor pode aceitar confirmação rápida; transferência de alto valor pode esperar maior segurança ou finalidade explícita.

12. Explorando uma Blockchain Didática - AP6

12.1 Objetivo do laboratório

Antes de construir uma blockchain completa, vamos observar propriedades básicas: hash, encadeamento, adulteração e validação. O objetivo é produzir evidência experimental de que a integridade do encadeamento é detectável, mas também compreender que isso ainda não resolve consenso entre múltiplos nós.

explorar_cadeia.py

```
from dataclasses import dataclass, asdict
import hashlib, json

@dataclass
class Bloco:
    indice: int
    dados: str
    hash_anterior: str
    hash: str = ""

    def calcular_hash(self):
        base = {"indice": self.indice, "dados": self.dados, "hash_anterior": self.hash_anterior}
        raw = json.dumps(base, sort_keys=True).encode()
        return hashlib.sha256(raw).hexdigest()

def novo(indice, dados, anterior):
    b = Bloco(indice, dados, anterior)
    b.hash = b.calcular_hash()
    return b

cadeia = [novo(0, "genesis", "0" * 64)]
for dado in ["A->B:10", "B->C:3", "C->D:1"]:
    cadeia.append(novo(len(cadeia), dado, cadeia[-1].hash))

def validar(cadeia):
    for i, b in enumerate(cadeia):
        if b.hash != b.calcular_hash():
            return False, f"hash inválido no bloco {i}"
        if i > 0 and b.hash_anterior != cadeia[i-1].hash:
            return False, f"ligação inválida no bloco {i}"
    return True, "cadeia válida"

print(validar(cadeia))
cadeia[1].dados = "A->B:1000"
print(validar(cadeia))
```

AP6 - Exploração de integridade, replicação e consenso

Atividade prática
Partindo da implementação didática, explore propriedades de hash e encadeamento e simule duas cópias divergentes da cadeia. O foco é explicar o que a criptografia resolve e o que ainda depende de um protocolo distribuído.

Objetivos da atividade

- Observar avalanche de hash e detecção de adulteração.
- Simular réplicas divergentes.
- Distinguir validação local de consenso global.
- Relacionar o experimento aos modelos de falha.

Requisitos obrigatórios

- Executar a cadeia didática.
- Criar pelo menos 20 blocos com dados gerados.
- Alterar bloco intermediário e registrar impacto.
- Criar duas cópias que recebem blocos diferentes.
- Propor uma regra de reconciliação e discutir limitações.

Roteiro mínimo

1. Registre hashes de referência.
2. Altere 1 caractere e compare saída.
3. Adultere um bloco antigo.
4. Recalcule hashes de blocos posteriores e observe que uma cadeia adulterada pode ser internamente consistente se o atacante puder reescrever tudo localmente.
5. Simule duas réplicas e proponha mecanismo de escolha.

Entregáveis

- Código executável.
- Tabela de experimentos.
- Capturas ou logs dos hashes.
- Texto curto: “hash não é consenso”.
- Diagrama das duas réplicas divergentes.

Questões para análise

1. Por que recalcular todos os hashes pode “consertar” localmente uma cadeia adulterada?

2. O que impede um participante de distribuir sua versão adulterada em uma rede real?

3. Que informação adicional um protocolo de consenso precisa considerar?

4. Qual relação existe entre fork e concorrência de eventos?

Critérios de avaliação

Tabela 20 - Critérios de avaliação da AP6

Critério	Evidência esperada
Experimentação	Manipulações e réplicas realmente executadas.
Conceitos	Hash, integridade e consenso corretamente distinguidos.
Análise	Limitações do modelo didático são explícitas.
Evidências	Resultados organizados e reproduzíveis.